Unidad 1. Orquestador concurrente de modelos

Franco Castro · AI Engineering · Ejercicio opcional

Qué es esto

Este ejercicio no llama a OpenAI, Anthropic ni a ningún otro servicio. Las demoras son `asyncio.sleep`: imitan la espera de la red y no frenan el programa. No se usa `time.sleep`.

Hay dos partes, porque la consigna pide las dos cosas a la vez: que las tareas arranquen juntas, y que un tope deje correr solo dos de diez.

- Parte 1. Tres corrutinas (`gpt_4_call`, `claude_3_call`, `local_llama_call`) se disparan con `asyncio.gather`. Un `asyncio.timeout` de 2 segundos cubre esa ejecución. La tercera duerme 3 segundos, así que aparece `TimeoutError`. El programa lo captura y sigue.
- Parte 2. Diez simulaciones se encolan con otro `gather`. Un `asyncio.Semaphore(2)` deja activas solo dos. En el log, "en curso" nunca pasa de 2.

Código

Archivo `unidad_1/orquestador.py`. Se corre con Python 3.12: `python unidad_1/orquestador.py`.

```
"""Ejercicio opcional de la Unidad 1: orquestador concurrente de modelos.

No llama a ninguna API. Las esperas son asyncio.sleep, que simula la red
sin bloquear el programa. Hay dos partes:

1. Tres modelos se disparan juntos con asyncio.gather. Un timeout de 2
   segundos corta la ejecución total si alguna tarda de más.
2. Diez simulaciones se encolan, y un semáforo deja correr solo 2 a la vez.
"""

import asyncio
import time

INICIO = time.perf_counter()


def marca() -> str:
    """Segundos desde que arrancó el programa, para leer el orden en el log."""
    return f"{time.perf_counter() - INICIO:5.2f}s"


async def gpt_4_call() -> str:
    print(f"[{marca()}] gpt_4_call inició", flush=True)
    await asyncio.sleep(0.4)
    print(f"[{marca()}] gpt_4_call terminó", flush=True)
    return "respuesta de gpt-4"


async def claude_3_call() -> str:
```

```
    print(f"[{marca()}] claude_3_call inició", flush=True)
    await asyncio.sleep(0.8)
    print(f"[{marca()}] claude_3_call terminó", flush=True)
    return "respuesta de claude"


async def local_llama_call() -> str:
    print(f"[{marca()}] local_llama_call inició", flush=True)
    await asyncio.sleep(3.0)
    print(f"[{marca()}] local_llama_call terminó", flush=True)
    return "respuesta de llama"


async def tres_en_paralelo() -> None:
    """Dispara los tres modelos a la vez y corta todo el grupo a los 2 segundos."""
    print("Parte 1. Tres modelos con gather. Timeout de 2 segundos sobre el total.", flush=True)
    try:
        async with asyncio.timeout(2.0):
            await asyncio.gather(
                gpt_4_call(),
                claude_3_call(),
                local_llama_call(),
            )
    except TimeoutError:
        print(f"[{marca()}] TimeoutError: la ejecución superó los 2 segundos.", flush=True)
        print("El programa no se detiene. Sigue con el semáforo.", flush=True)


async def simulacion(numero: int, semaforo: asyncio.Semaphore, en_curso: list[int]) -> None:
    """Una llamada falsa. El semáforo impide que haya más de 2 activas."""
    async with semaforo:
        en_curso[0] += 1
        print(
            f"[{marca()}] simulación {numero:02d} inicia | en curso: {en_curso[0]}",
            flush=True,
        )
        await asyncio.sleep(0.4)
        en_curso[0] -= 1
        print(
            f"[{marca()}] simulación {numero:02d} termina | en curso: {en_curso[0]}",
            flush=True,
        )


async def lote_con_semaforo() -> None:
    """Diez llamadas encoladas. Solo dos pueden estar dentro del semáforo."""
    print(flush=True)
    print("Parte 2. Diez simulaciones. Semáforo de 2.", flush=True)
    semaforo = asyncio.Semaphore(2)
    en_curso = [0]
    await asyncio.gather(
        *(simulacion(numero, semaforo, en_curso) for numero in range(1, 11))
    )


async def main() -> None:
    await tres_en_paralelo()
    await lote_con_semaforo()
    print(f"[{marca()}] Fin. El programa terminó por su cuenta.", flush=True)
```

```
if __name__ == "__main__":
    asyncio.run(main())
```

Salida real

Corrido con Python 3.12.15. Los tres modelos arrancan en el mismo instante (0.00s). `local_llama_call` iba a tardar 3 segundos y no llega a terminar: a los 2.01s se captura el timeout y el programa sigue. En la parte 2, “en curso” vale 1 o 2, nunca 3 o más.

```
Parte 1. Tres modelos con gather. Timeout de 2 segundos sobre el total.
[ 0.00s] gpt_4_call inició
[ 0.00s] claude_3_call inició
[ 0.00s] local_llama_call inició
[ 0.40s] gpt_4_call terminó
[ 0.80s] claude_3_call terminó
[ 2.01s] TimeoutError: la ejecución superó los 2 segundos.
El programa no se detiene. Sigue con el semáforo.

Parte 2. Diez simulaciones. Semáforo de 2.
[ 2.01s] simulación 01 inicia | en curso: 1
[ 2.01s] simulación 02 inicia | en curso: 2
[ 2.41s] simulación 01 termina | en curso: 1
[ 2.41s] simulación 02 termina | en curso: 0
[ 2.41s] simulación 03 inicia | en curso: 1
[ 2.41s] simulación 04 inicia | en curso: 2
[ 2.81s] simulación 03 termina | en curso: 1
[ 2.81s] simulación 04 termina | en curso: 0
[ 2.81s] simulación 05 inicia | en curso: 1
[ 2.81s] simulación 06 inicia | en curso: 2
[ 3.21s] simulación 05 termina | en curso: 1
[ 3.21s] simulación 06 termina | en curso: 0
[ 3.21s] simulación 07 inicia | en curso: 1
[ 3.21s] simulación 08 inicia | en curso: 2
[ 3.61s] simulación 07 termina | en curso: 1
[ 3.61s] simulación 08 termina | en curso: 0
[ 3.61s] simulación 09 inicia | en curso: 1
[ 3.61s] simulación 10 inicia | en curso: 2
[ 4.01s] simulación 09 termina | en curso: 1
[ 4.01s] simulación 10 termina | en curso: 0
[ 4.01s] Fin. El programa terminó por su cuenta.
```

Criterios

- No hay `time.sleep`. La espera es `await asyncio.sleep`.
- Las tres tareas de la parte 1 inician en el mismo instante.
- `TimeoutError` se captura. El proceso sigue y termina solo.
- De diez simulaciones, solo dos están en curso al mismo tiempo.